ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Challenge II — Feedback

Challenge 1 — Point-free workout

Key observations

- `negateAll` : we are *transforming* every element → `map` .
  The transformation is `negate` : it is a standard function from the prelude, but you can define it as, e.g. `(* (-1))`
- `longWords` : we are *selecting* elements by a condition → `filter` .
  The condition is "length greater than 4", which is the composition `(> 4) . length` .
- `countEven` : we are *selecting* then *combining* → `filter` then `length` .
  As a pipeline: `length . filter even` .

```
negateAll :: [Int] -> [Int]
negateAll = map negate

longWords :: [String] -> [String]
longWords = filter ((> 4) . length)

countEven :: [Int] -> Int
countEven = length . (filter even)
```

`longWords` is the trickiest: the predicate `(> 4) . length` first computes the length of a string, then tests whether it exceeds 4. The composition builds a new predicate from two pieces — a function `length :: String -> Int` and a section `(> 4) :: Int -> Bool` — without ever naming the string being tested.

As to `countEven` , just a note to point out that the point-free version traverses the list twice. A more efficient version is the following alternative:

```haskell
countEven = foldr (\x acc -> if even x then acc + 1 else acc) 0
```

Scala alternative. Attempting the same in Scala shows that the translation breaks down in all cases. Scala does not have sctions or lightweight composition.

```scala
// negateAll: no negate function, no sections → need a lambda, or placeholder syntax, use the fo
val negateAll = (xs: List[Int]) => xs.map(- _)

// longWords: no composition operator for inline use → need a lambda or placeholder syntax. Use
val longWords = (_: List[String]).filter(_.length > 4)

// countEven: no point-free pipeline → need to name the argument
val countEven = (_: List[Int]).count(_ % 2 == 0)
```

The Scala versions are concise thanks to `_`, but they are not point-free: the list argument must be named (or appear as a method receiver). The `_` placeholder works well inline inside a method call: notice, instead, that the placeholder occurrences as method receivers need typing annotations. This is a consequence of Scala's somewhat limited inference capabilities.

Challenge 2 — Flip at work

```haskell
flip :: (a -> b -> c) -> b -> a -> c
flip f y x = f x y
```

We first look at `prepend`

We want a function that, given a list and an element, appends the element at the end. That's `\xs x -> xs ++ [x]`.

To make it point free it is tempting to use `flip (++)` but the type is `[a] -> [a] -> [a]`, so that will not work directly: we need to wrap the secon argument into a singleton list before being able to use the flipped version of `++`.

```haskell
flip (\x -> (++ [x]))
```

To see why this works observe that `\x -> (++ [x]) :: a -> [a] -> [a]` does the right thing, but receives the arguments in the wrong order: with `flip` we restore the correct order.

Now, to make it point free, observe that `\x -> (++ [x])` can be expressed point-free as follows `(flip (++)) . (:[])`. In fact:

- `(:[])` is the point-free version of `\x -> x:[]` that is `\x -> [x]`
- `(flip (++))` swaps it arguments
- so `(flip (++)) . (:[])` first wraps its (first) argument in a singleton and the `(flip (++))` does the swap (as in `(++ [x]))

Fiinalizing we have

```
flip ((flip (++)) . (:[]))
```

Having gone through the various steps, it is only fair to say that the version with lambda reads a lot better. Nothwistanding that, the exercise is useful to gain a tigher grip on the point-free syntax.

Now `reverse : flip` is more useful here.

The `foldr` version :

```
reverse = foldr (\x acc -> acc ++ [x]) []
```

This is O(n²) because `(++)` is linear in its left argument, and the accumulator grows at each step.

The `foldl` version using `flip` :

```
reverse = foldl (flip (:)) []
```

Why it works: `foldl` passes `(accumulator, element)` to the combining function. We want to cons the element onto the accumulator: `(:) element accumulator`. But `(:)` takes the element first and the list second, while `foldl` supplies them in the opposite order (accumulator first, element second). `flip (:)` swaps the arguments, giving us exactly what we need.

Why this is better: Deach step is a single cons operation (O(1)), so the whole fold is O(n). The `foldr` version appends to the end at each step (O(n) per step), giving O(n²) overall.

Trace on `[1, 2, 3]` :

```
foldl (flip (:)) []       [1, 2, 3]
foldl (flip (:)) [1]      [2, 3]       -- flip (:) [] 1  =  1 : []  =  [1]
foldl (flip (:)) [2, 1]   [3]          -- flip (:) [1] 2 =  2 : [1] =  [2,1]
foldl (flip (:)) [3, 2, 1] []          -- flip (:) [2,1] 3 = 3 : [2,1] = [3,2,1]
= [3, 2, 1]
```

Challenge 3 — Lifting

```
lift :: (b -> b -> b) -> (a -> b) -> (a -> b) -> (a -> b)
lift op f g = \x -> op (f x) (g x)
```

The function applies both `f` and `g` to the same argument `x`, then combines the results with `op`. This is the pointwise lifting of a binary operation from values to functions.

```
adder      = lift (+)
multiplier = lift (*)
divider    = lift (/)
```

Verification: `lift (+) (*3.0) (*5.0) $ 2 = (+) ((*3.0) 2) ((*5.0) 2) = (+) 6.0 10.0 = 16.0` ✓

Elixir alternative:

```
lift = fn op, f, g ->
  fn x -> op.(f.(x), g.(x)) end
end

adder      = fn f, g -> lift.(&+/2, f, g) end
multiplier = fn f, g -> lift.(&*/2, f, g) end
divider    = fn f, g -> lift.(&//2, f, g) end
```

Verification `lift.(&+/2, fn x -> x * 3.0 end, fn x -> x * 5.0 end).(2) # 16.0`

The logic is identical. The main differences are syntactic: anonymous functions require `fn...end`, calling them requires a dot (`f.(x)`), and operators must be *captured* with `&+/2`.

Challenge 4 — Meet and meetAll

`meet` via `lift`:

```haskell
meet :: (Int -> Bool) -> (Int -> Bool) -> (Int -> Bool)
meet = lift (&&)
```

This is immediate: `lift (&&) f g` returns a predicate that is true when both `f` and `g` are true. The `(&&)` is just the operator being lifted.

`liftAll`:

We need to combine a list of functions using a binary operation and a base value. This is exactly what `foldr` does — but on *functions* rather than on plain values: e.g.

```haskell
liftAll :: (b -> b -> b) -> (a -> b) -> [a -> b] -> (a -> b)
liftAll op b fs = foldr (lift op) b fs
```

This reads: fold the list of functions using `lift op` as the combining operation and `b` as the base case. At each step, `lift op` takes the current function and the accumulated function and produces a new function that applies both and combines the results with `op`. Now we can write

```haskell
liftAll (+) (const 0) [f1, f2, f3]` returns a function that sums the results of all three on any
```

Note: the class version of the challenge was proposed with a different type:

```haskell
liftAll :: (b -> b -> b) -> b -> [a -> b] -> (a -> b)
```

That's admittedly clumsy, as the fallback case, when the `[a -> b]` list is empty, should be a function of the same type, not `b`. With this type to answer to the challenge you can cope as follows:

```haskell
liftAll op b fs = foldr (lift op) (const b) fs
```

`meetAll`:

With `liftAll :: (b -> b -> b) -> (a -> b) -> [a -> b] -> (a -> b)`, the definition is simply:

```haskell
meetAll :: [Int -> Bool] -> (Int -> Bool)
meetAll = liftAll (&&) (const True)
```

Verification:

```
meetAll [even, (>0), (<100)] 42 = even 42 && (>0) 42 && (<100) 42 = True && True && True = True
```

And: `meetAll [even, (>0), (<100)] 43 = even 43 && ... = False && ... = False`

Elixir alternative:

```elixir
meet_all = fn predicates ->
  fn x ->
    Enum.reduce(predicates, true, fn p, acc -> acc and p.(x) end)
  end
end

# Test:
meet_all.([&Integer.is_even/1, fn x -> x > 0 end, fn x -> x < 100 end]).(42)
# true
```

The structure is the same: reduce the list of predicates with `and`, starting from `true`. The dot-call syntax (`p.(x)`) and the explicit `fn...end` wrappers make the code more verbose than Haskell's, but the *pattern* — fold a list of functions with a lifted binary operator — is identical.

Challenge 5 — Church numerals

- `zero f x = x` — applying `f` zero times means doing nothing: return `x` unchanged.
- `one f x = f x` — apply `f` once.
- `two f x = f (f x)` — apply `f` twice.
- `three f x = f (f (f x))` — apply `f` three times.

```haskell
type Church a = (a -> a) -> a -> a

zero, one, two, three :: Church a
zero  f x = x
one   f x = f x
two   f x = f (f x)
three f x = f (f (f x))
```

`toInt :` we need to convert a Church numeral to an integer. A Church numeral applies `f` some number of times to `x`. If we choose `f = (+1)` and `x = 0`, the result is exactly the number of

applications:

```
toInt :: Church Int -> Int
toInt n = n (+1) 0
```

Verification: `toInt zero = zero (+1) 0 = 0` ✓, `toInt three = three (+1) 0 = (+1)(+1)(+1) 0 = 3` ✓

`fromInt`: convert an integer `n` to the Church numeral that applies `f` n-times. Using direct recursion:

```
fromInt :: Int -> Church a
fromInt 0 f x = x
fromInt n f x = f (fromInt (n - 1) f x)
```

Or, using the iterate-and-compose pattern:

```
fromInt n f = foldr (.) id (replicate n f)
```

Here `replicate n f` creates a list `[f, f, ..., f]` with `n` copies of `f`, which are composed as `f . f . f ... . f . id` by folding.

`churchAdd`: if `m` applies `f` m-times and `n` applies `f` n-times, then m+n applications means: apply `n` first, then `m`:

```
churchAdd :: Church a -> Church a -> Church a
churchAdd m n f = (m f) . (n f)
```

`churchMul`: if `n f` is "apply `f` n-times" (a single function `a -> a`), then `m (n f)` applies that function m-times, giving m×n applications total. We don't need to mention `f`, in fact:

```
churchMul :: Church a -> Church a -> Church a
churchMul m n = m . n
```

Verification:

```
toInt (churchAdd two three)  ==  toInt (\f x -> two f (three f x))  ==  5  ✓
toInt (churchMul two three)  ==  toInt (\f -> two (three f))        ==  6  ✓
```

Bonus: `churchExp`.

```
churchExp :: Church a -> Church (a -> a) -> Church a
churchExp m n = n m
```

This works because `n`, a Church numera, applies its argument n times: if the argument is `m`, the church numeral that applies its argument m times, the result of `n m` applies $n \times m$ time its argument, with is just the church numeral for m^n. .

Note that the type of `n` is `Church (a -> a)` rather than `Church a`. That is because `n` must iterate `m`. So `n` is iterating a function whose type is `(a -> a) -> (a -> a)` — that is, `n` operates on functions `a -> a`, not on plain values `a`. Hence `n :: Church (a -> a)`.